

Lecture 15

Introduction to NOSQL and Big Data

Objectives

- [illegible]

Introduction to NOSQL Systems

- **Emergence of NOSQL Systems**
- **Characteristics of NOSQL Systems**
- **Categories of NOSQL Systems**

Emergence of NOSQL Systems

- SQL system may not be appropriate for some applications such as Emails
 - SQL systems offer too many services (powerful query language, concurrency control, etc.), which this application may not need;
 - structured data model such the traditional relational model may be too restrictive.
 - SQL require schemas, which are not required by many of the NOSQL systems.

Emergence of NOSQL Systems

- Examples of NOSQL systems:
 - Google – BigTable
 - Amazon – DynamoDB
 - Facebook – Cassandra
 - MongoDB
 - CouchDB
 - Graph databases like Neo4J and GraphBase

Characteristics of NOSQL Systems

- NOSQL characteristics related to distributed databases and distributed systems.
- NOSQL characteristics related to data models and query languages.

characteristics related to distributed databases and distributed systems

1- Scalability:

- horizontal scalability: adding more nodes for data storage and processing as the volume of data grows.
- Vertical scalability: expanding the storage and computing power of existing nodes.
- In NOSQL systems, horizontal scalability is employed while the system is operational, so techniques for distributing the existing data among new nodes without interrupting system operation are necessary.

characteristics related to distributed databases and distributed systems

2- Availability, Replication and Eventual Consistency:

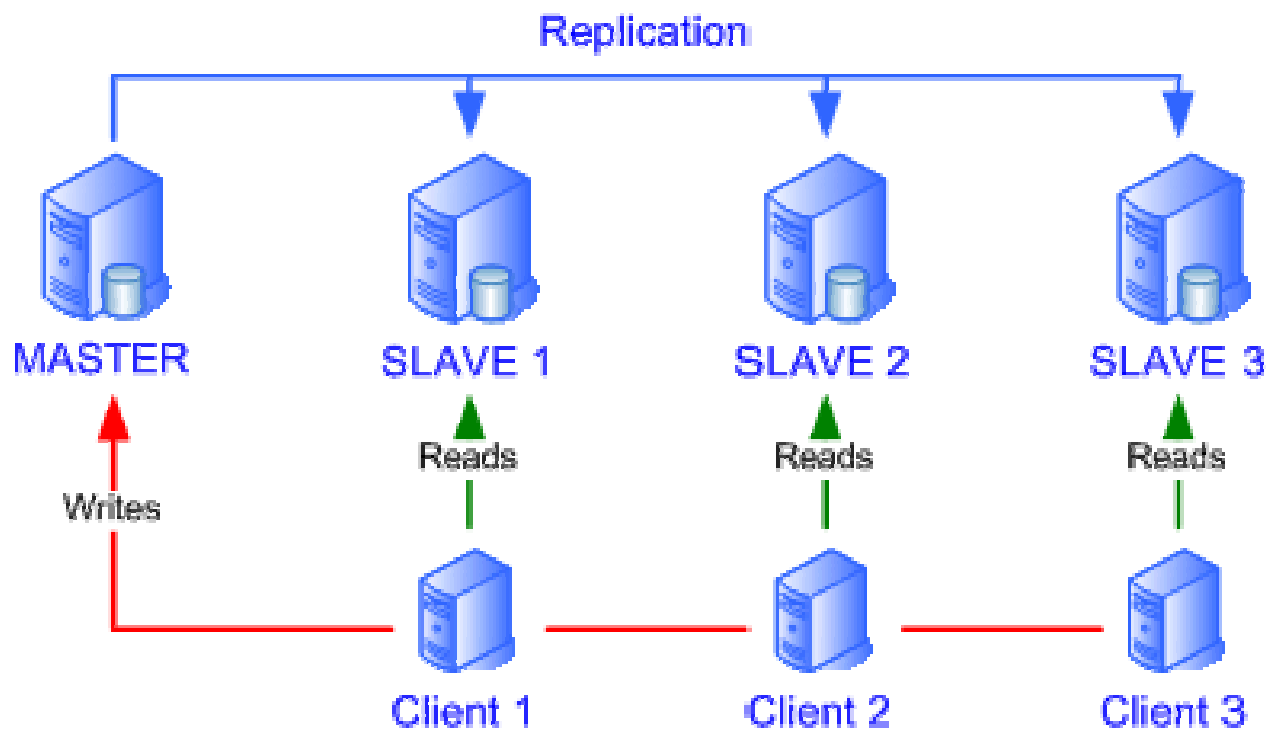
- Data is replicated over two or more nodes in a transparent manner.
- Update must be applied to every copy of the replicated data items.
- Eventual consistency: is a consistency model used in distributed computing to achieve high availability that informally guarantees that, if no new updates are made to a given data item, eventually all accesses to that item will return the last updated value.

characteristics related to distributed databases and distributed systems

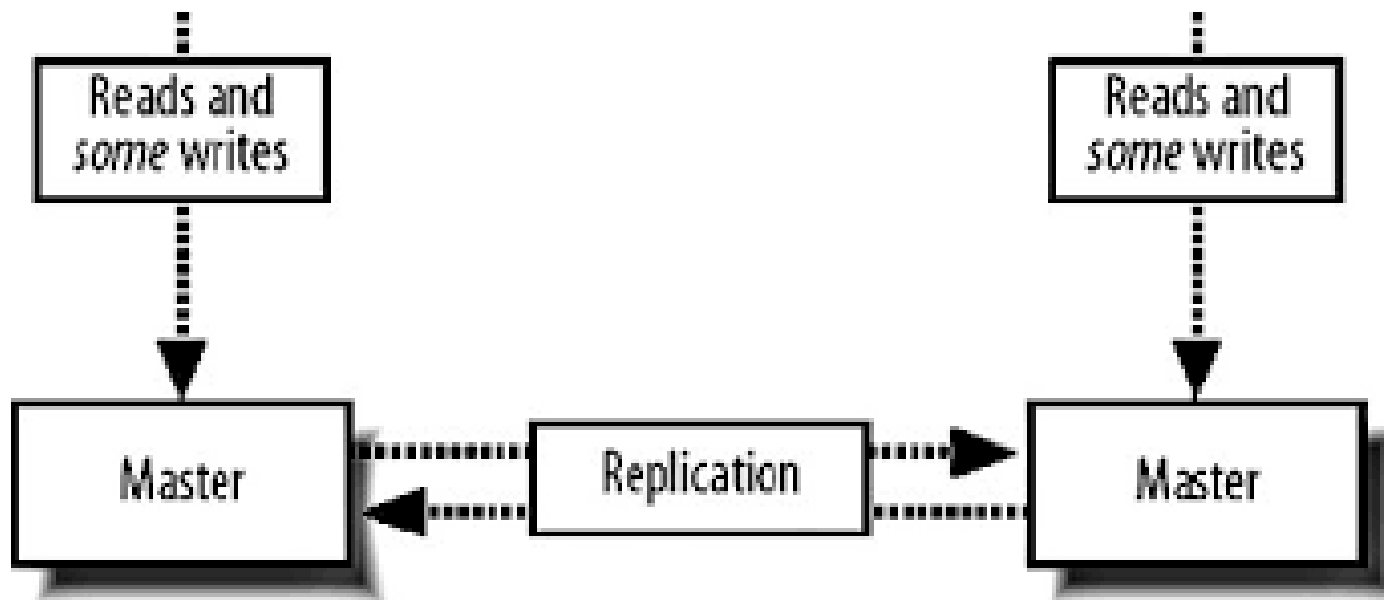
3- Replication Models:

- **Master-slave replication:** requires one copy to be the master copy;
 - Write operations must be applied to the master copy, usually using eventual consistency
 - For read, all reads are from the master copy, or reads at the slave copies but would not guarantee that the values are the latest writes.
- **Master-master replication:** allows reads and writes at any of the replicas.
 - The values of an item will be temporarily inconsistent.
 - Reconciliation method to resolve conflicting write operations of the same data item at different nodes must be implemented as part of the master-master replication scheme.

characteristics related to distributed databases and distributed systems



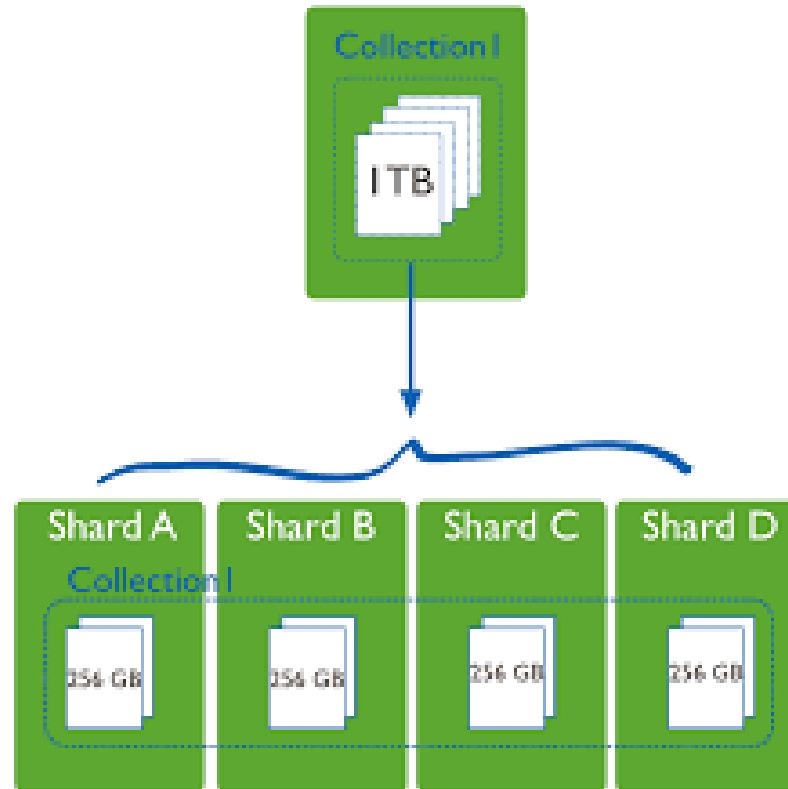
characteristics related to distributed databases and distributed systems



characteristics related to distributed databases and distributed systems

- 4- Sharding of Files:
 - Files can have many millions of records accessed concurrently by thousands of users.
 - Sharding (also known as horizontal) serves to distribute the load of accessing the file records to multiple nodes.
 - Shards works in tandem to improve load balancing on the replication as well as data availability.

characteristics related to distributed databases and distributed systems



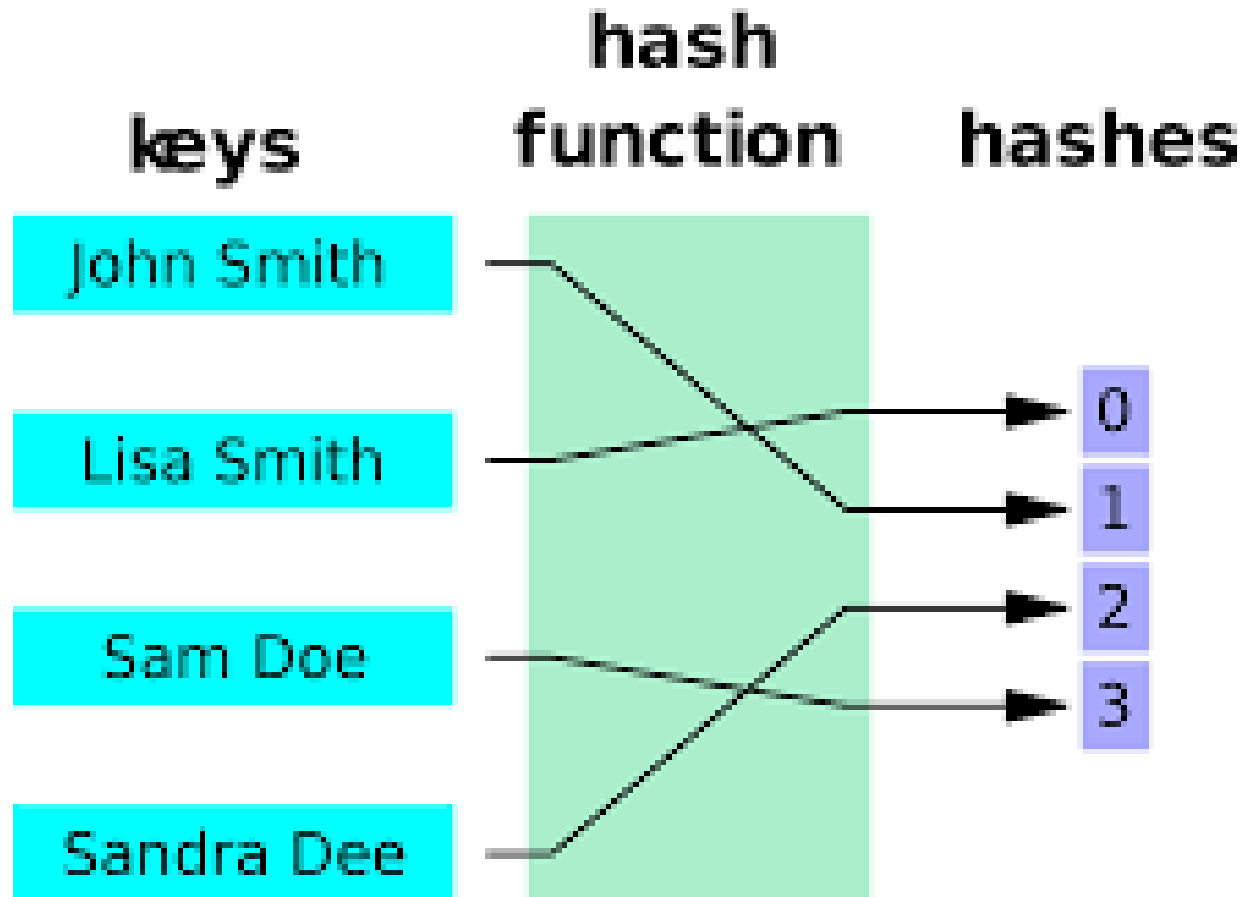
characteristics related to distributed databases and distributed systems

•5- High-Performance Data Access:

- **Hashing:** The location of the value is given by the result of $h(k)$.
- **Range partitioning:** the location is determined via a range of key values. *Example: location i would hold the objects whose key values K are in the range $K_{\min} \leq K \leq K_{\max}$.*

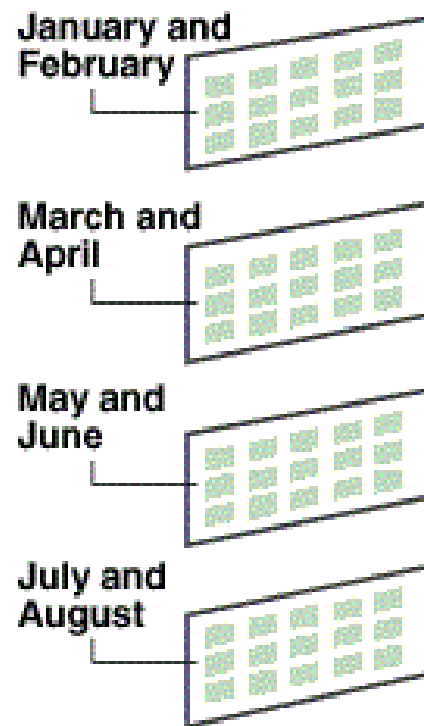
In applications that require range queries, where multiple objects within a range of key values are retrieved, range partitioned is preferred.

characteristics related to distributed databases and distributed systems

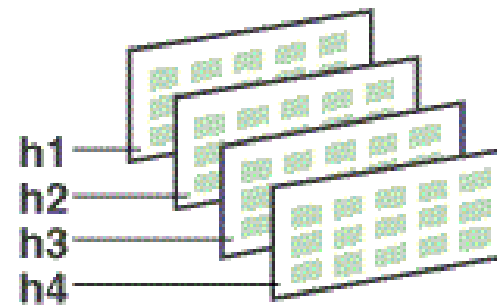


characteristics related to distributed databases and distributed systems

**Range
Partitioning**



**Hash
Partitioning**



characteristics related to data models and query languages.

- 1- Not Requiring a Schema:
 - Allowing semi-structured and self describing data.
 - The users can specify a partial schema in some systems to improve storage efficiency, but it is not required to have a schema in most of the NOSQL systems.
 - Constraints on the data would have to be programmed in the application programs that access the data items.
 - Languages for describing semi-structured data: JSON (JavaScript Object Notation) and XML (Extensible Markup Language)

characteristics related to data models and query languages.

- 2- Less Powerful Query Languages:

- In many applications that use NOSQL systems may not require a powerful query language such as SQL, because search (read) queries in these systems often locate single objects in a single file based on their object keys.
- Reading and writing the data objects is accomplished by calling the appropriate operations by the programmer (API).
- SCRUD: Search, Create, Read, Update and Delete
- Provide a high-level query language, but it may not have the full power of SQL, for example the joins need to be implemented in the application programs.

characteristics related to data models and query languages.

- 3- Versioning:
 - Provide storage of multiple versions of the data items, with the timestamps of when the data version was created.

Categories of NOSQL Systems

The most common categories:

1. Document-based NOSQL systems:

- Store data in the form of documents using well-known formats such as JSON.
- Documents are accessible via their document id, but can also be accessed rapidly using other indexes.

2. NOSQL key-value stores:

- Fast access by the key to the value associated with the key
- Value can be a record or an object or a document or even have a more complex data structure.

3. Column-based or wide column NOSQL systems:

- Partition a table by column into column families
- Form of vertical partitioning.

4. Graph-based NOSQL systems:

- Data is represented as graphs
- Related nodes can be found by traversing the edges using path expressions.

Categories of NOSQL Systems

Additional categories :

5. Hybrid NOSQL systems:

- These systems have characteristics from two or more of the common categories..

6. Object databases.

7. XML databases.

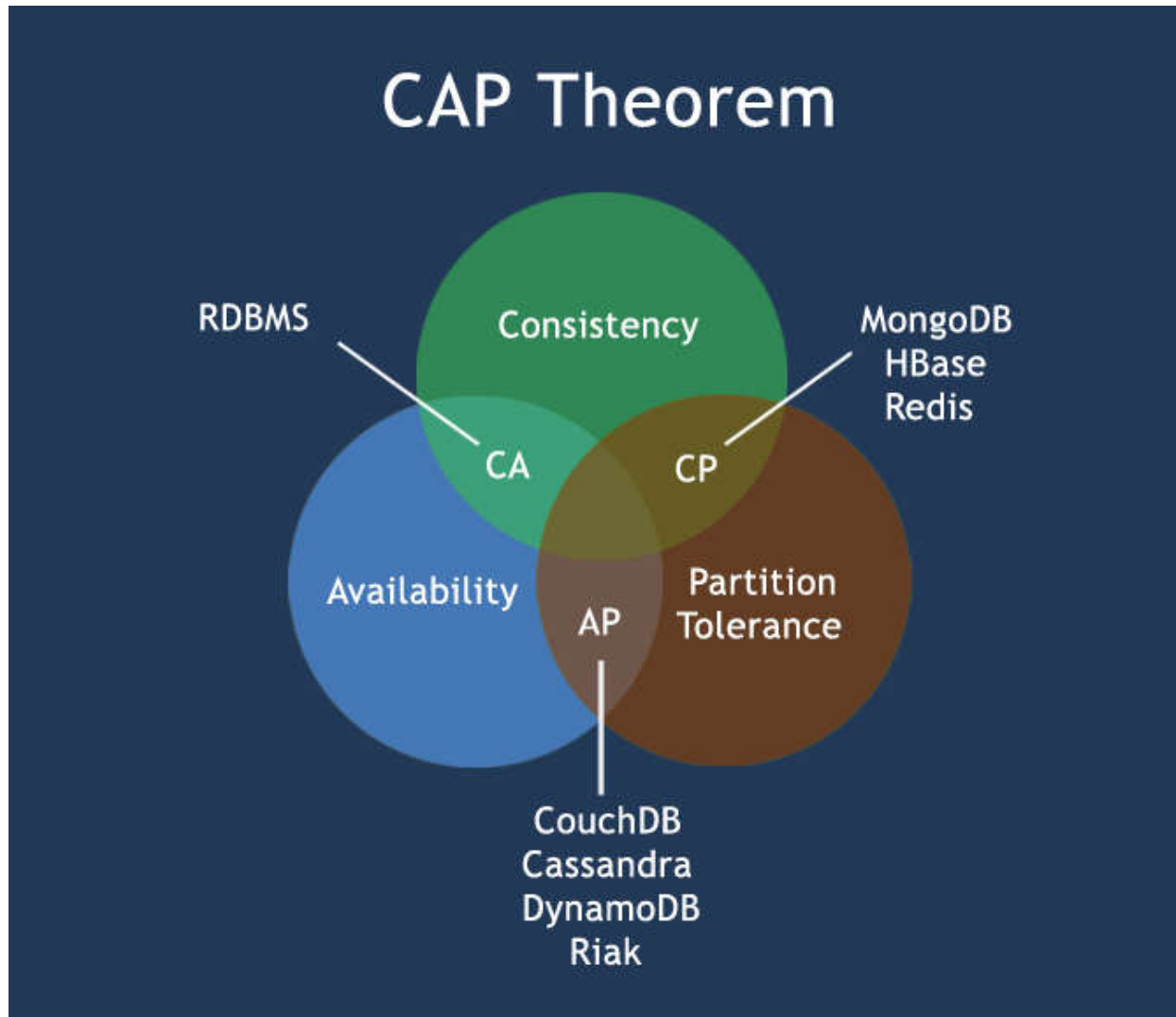
The CAP Theorem

- The CAP: it's impossible to guarantee consistency, availability and partition tolerance at the same time in a distributed system with data replication.
- Two properties out of the three to guarantee.
- Weaker consistency levels are often used in NOSQL system instead of guaranteeing serializability.
- Eventual consistency is used.

The CAP Theorem

- The CAP theorem is used to explain some of the competing requirements in a distributed system with replication.
- The three letters in CAP refers to
 - **Consistency** (among replicated copies):
 - The nodes will have the same copies of a replicated data item visible for various transactions.
 - **Availability** (of the system for read and write operations) :
 - Each read or write will either be processed successfully or will receive a message that the operation cannot be completed.
 - **Partition tolerance** (in the face of the nodes in the system being partitioned by a network fault).:
 - The system can continue operating if the network connecting the nodes has a fault that results in two or more partitions,
 - Nodes in each partition can only communicate among each other.

The CAP Theorem



Introduction

MongoDB Data Model

MongoDB CRUD Operations

MongoDB Distributed Systems Characteristics

- Document-based NOSQL systems store data as collections of similar documents.
- Documents resemble complex objects or XML documents
- Documents in a collection should be similar, but they can have different attributes.
- Document-based NOSQL systems: MongoDB and CouchDB.

MongoDB Data Model

- MongoDB is a free and open-source cross-platform document-oriented database.
- Classified as a NoSQL database,



MongoDB Data Model

- MongoDB documents are stored in BSON (Binary JSON) format.
- BSON is a variation of JSON with some additional data types and is more efficient for storage than JSON.
- Individual documents are stored in a collection.
- The operation ***createCollection*** is used to create each collection.

MongoDB Data Model

- Example: create a collection called project to hold PROJECT objects from the COMPANY database :

```
db.createCollection("project", { capped : true, size :  
1310720, max : 500 } )
```

- “project” is the name of the collection (**Mandatory**)
- Capped: capped means it has upper limits on its storage space (size) and number of documents (max).
- Capping helps the system to choose the storage options for each collection.

MongoDB Data Model

- Example: create a document collection called worker :
db.createCollection("worker", { capped : true, size : 5242880, max : 2000 })
- Each document has a unique ObjectId field “_id”
- The _id is by default:
 - Automatically indexed in the collection.
 - The value is system-generated.
 - System-generated have a specific format – “combines the timestamp when the object is created, the node id, the process id and a counter “.
 - User-generated can have any value specified by the user as long as its.

MongoDB Data Model

- A collection does not have a schema.
- The structure of the data fields in documents is chosen based on how documents will be accessed and used, and the user can choose a normalized design (similar to normalized relational tuples) or a denormalized design (similar to XML documents or complex objects).
- Interdocument references can be specified by storing in one document the ObjectId or ObjectIds of other related documents.

MongoDB Data Model

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1985-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888885555	5
Alicia	J	Zelaya	000887777	1988-01-10	3321 Castle, Spring, TX	F	25000	087654321	4
Jennifer	S	Wallace	087654321	1941-08-20	201 Berry, Bellairs, TX	F	43000	888885555	4
Ramesh	K	Narayan	888884444	1982-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	087087087	1969-03-20	980 Dallas, Houston, TX	M	25000	087654321	4
James	E	Borg	888885555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Company database example

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1989-05-22
Administration	4	087654321	1995-01-01
Headquarters	1	888885555	1981-08-10

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellairs
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
888884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
000887777	30	30.0
000887777	10	10.0
087087087	10	35.0
087087087	30	5.0
087654321	30	20.0
087654321	20	15.0
888885555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellairs	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alicia	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
087654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alicia	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

MongoDB Data Model

(a) project document with an array of embedded workers:

```
{
  _id:          "P1",
  Pname:        "ProductX",
  Plocation:    "Bellaire",
  Workers: [
    { Ename: "John Smith",
      Hours: 32.5
    },
    { Ename: "Joyce English",
      Hours: 20.0
    }
  ]
};
```

Project info

Embedded workers info

MongoDB Data Model

(b) project document with an embedded array of worker ids:

```
{
  _id: "P1",
  Pname: "ProductX",
  Plocation: "Bellaire",
  WorkerIds: [ "W1", "W2" ]
}

{ _id: "W1",
  Ename: "John Smith",
  Hours: 32.5
}

{ _id: "W2",
  Ename: "Joyce English",
  Hours: 20.0
}
```

Project info

Embedded workers array

Workers

MongoDB Data Model

(c) normalized project and worker documents (not a fully normalized design for M:N relationships):

```
{
  _id:      "P1",
  Pname:    "ProductX",
  Plocation: "Bellaire"
}
{
  _id:      "W1",
  Ename:    "John Smith",
  ProjectId: "P1",
  Hours:    32.5
}
{
  _id:      "W2",
  Ename:    "Joyce English",
  ProjectId: "P1",
  Hours:    20.0
}
```

Project ID as an attribute

3.2 MongoDB Data Model

(d) inserting the documents in (c) into their collections "project" and "worker":

```
db.project.insert( { _id: "P1", Pname: "ProductX", Plocation: "Bellaire" } )
```

```
db.worker.insert( [ { _id: "W1", Ename: "John Smith", ProjectId: "P1", Hours: 32.5 },  
                    { _id: "W2", Ename: "Joyce English", ProjectId: "P1",  
                      Hours: 20.0 } ] )
```

MongoDB CRUD Operations

- Insert:
 - `db.<collection_name>.insert(<document(s)>)`
 - Example:
 - `Db.project.insert({_id:"P1", Pname:"ProjectX",Plocation:"Jenin"})`
- Delete: remove
 - `db.<collection_name>.remove(<condition>)`
 - Example:
 - `db.project.remove( {"_id": ObjectId("P1")});`

MongoDB CRUD Operations

- Read: find
 - `db.<collection_name>.find(<condition>)`
 - Example:
 - `Db.project.find({"_id": ObjectId("P1")})`
- Update:
 - `db.<collection_name>. update(SELECTIOIN_CRITERIA, UPDATED_DATA)`
 - Example:
 - `Db.project.update({"_id" : ObjectId(P1)},{ $set:{'PLocation':'AAUJ'}})`

MongoDB Distributed Systems Characteristics

- Replication in MongoDB
- Sharding in MongoDB

Replication in MongoDB

- Master-slave approach for replication.
- All read and write are done on the primary copy.
- Secondary copies are to recover from primary fails.

Sharding in MongoDB

- Sharding of the documents in the collection—also known as horizontal partitioning—divides the documents into disjoint partitions known as shards.
- Two ways:
 - Range partitioning
 - Hash partitioning

Sharding in MongoDB

- Range and Hash partitioning require that the user specify a particular document field to be used as the basis for partitioning the documents into shards.
- The partitioning field—known as the “*shard key*”, must exist in every document in the collection, and it must have an index.
- The values of the shard key are divided into *chunks*, and the documents are partitioned based on the chunks of shard key values

Sharding in MongoDB

- Chunks created by specifying a range of key values and each chunk contains the key values in one range.
- If range queries are commonly applied to a collection (for example, retrieving all documents whose shard key value is between 200 and 400), then range partitioning is preferred
 - Because each range query will typically be submitted to a single node that contains all the required documents in one shard.
- If most searches retrieve one document at a time, hash partitioning may be preferable because it randomizes the distribution of shard key values into chunks.

Sharding in MongoDB

- MongoDB queries are submitted to a module called the *query router*, which keeps track of which nodes contain which shards based on the particular partitioning method used on the shard keys.
 - The query will be routed to the nodes that contain the shards that hold the documents that the query is requesting.
 - If the system cannot determine which shards hold the required documents, the query will be submitted to all the nodes that hold shards of the collection.

Sharding in MongoDB

- Sharding and replication are used together:
 - Sharding focuses on improving performance via load balancing and horizontal scalability.
 - Replication focuses on ensuring system availability when certain nodes fail in the distributed system.

Why nosql?

- Document or table ?

ISBN	title	author	format	price
9780992461225	JavaScript: Novice to Ninja	Darren Jones	ebook	29.00
9780994182654	Jump Start Git	Shaumik Daityari	ebook	29.00

```
{  
  ISBN: 9780992461225,  
  title: "JavaScript: Novice to Ninja",  
  author: "Darren Jones",  
  format: "ebook",  
  price: 29.00  
}
```

Why nosql?

- Alter the table and add Description, Rate and Reviews
- NOSQL is Flexible

```
{  
  ISBN: 9780992461225,  
  title: "JavaScript: Novice to Ninja",  
  author: "Darren Jones",  
  year: 2014,  
  format: "ebook",  
  price: 29.00,  
  description: "Learn JavaScript from scratch!",  
  rating: "5/5",  
  review: [  
    { name: "A Reader", text: "The best JavaScript book I've ever read." },  
    { name: "JS Expert", text: "Recommended to novice and expert developers alike." }  
  ]  
}
```

No Schema restrictions

Why nosql?

- SQL is Restricted !

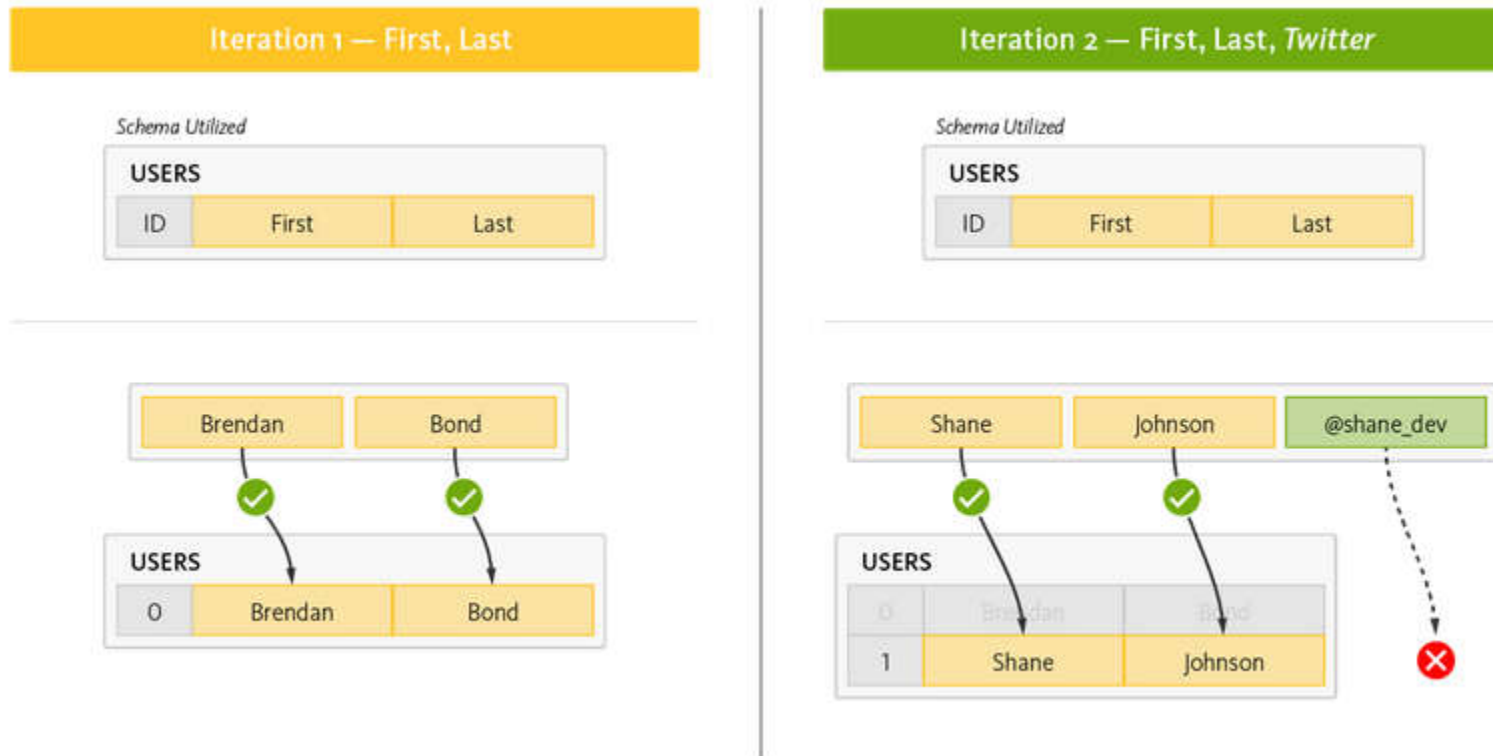
```
[-] ALTER TABLE Books
    ADD [Description] varchar
    Add Rate  varchar

[-] CREATE TABLE Reviews
(
    ID int,
    Name varchar(255),
    [text] varchar(255),
);
```

Fill the data

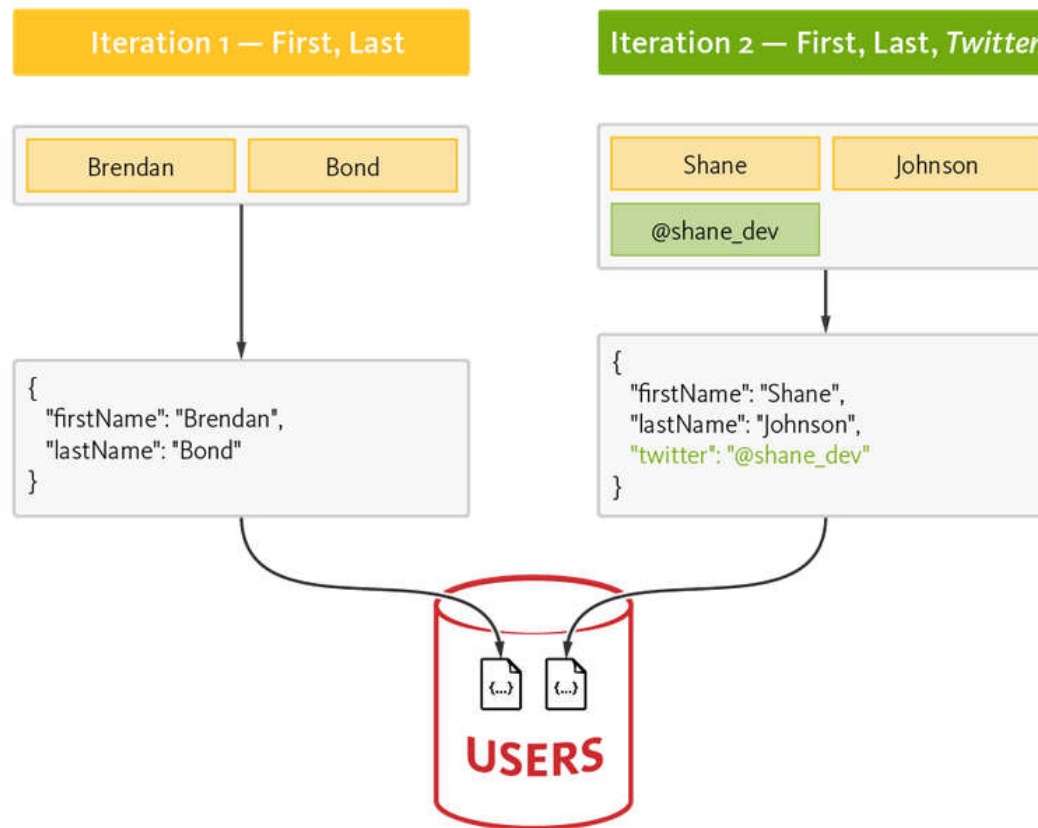
Why nosql? - Use cases where NoSQL will outperform SQL

- Agile - Flexibility for Faster Development



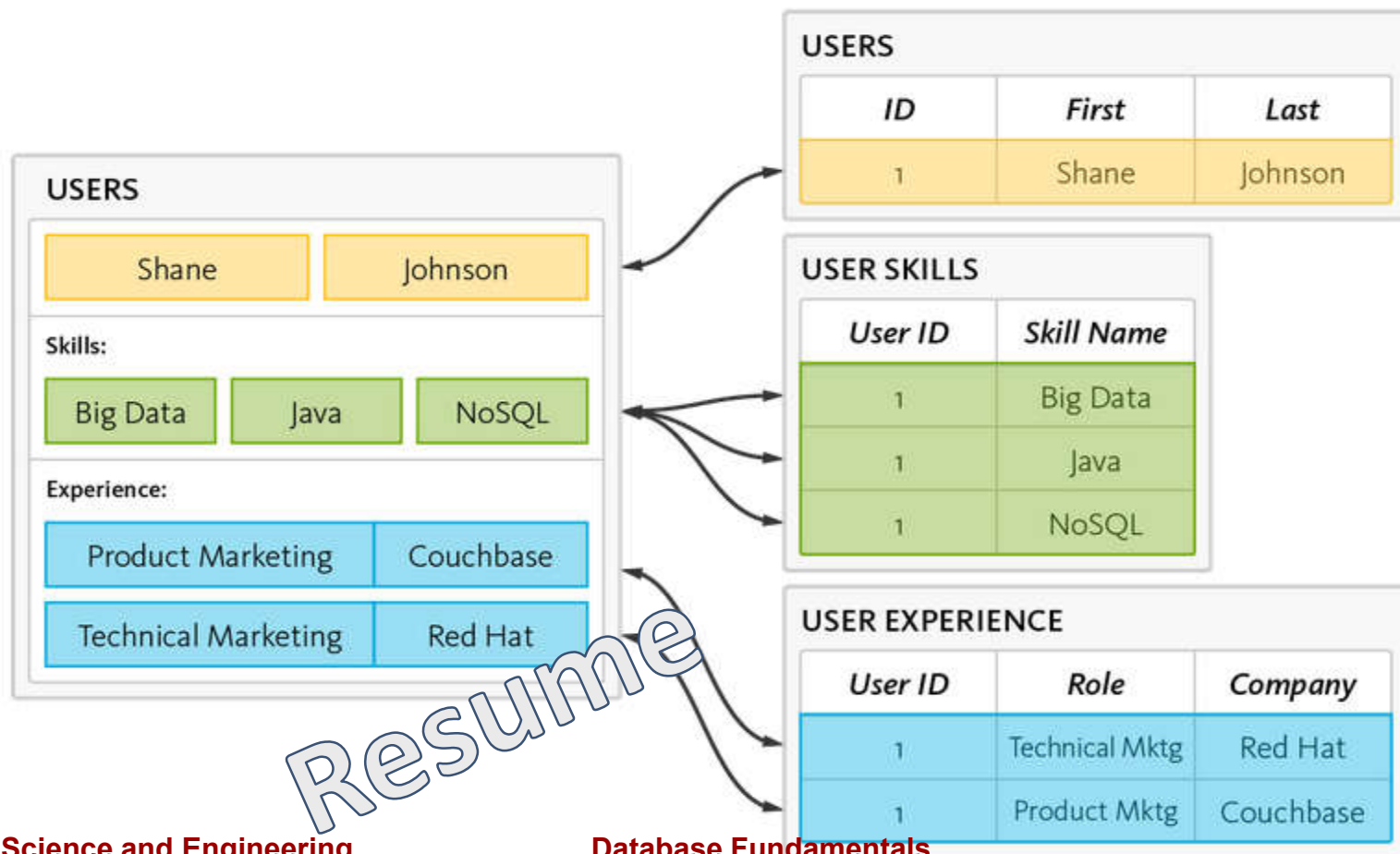
Why nosql? - Use cases where NoSQL will outperform SQL

- Agile - Flexibility for Faster Development



Why nosql? - Use cases where NoSQL will outperform SQL

- Agile - Simplicity for Easier Development



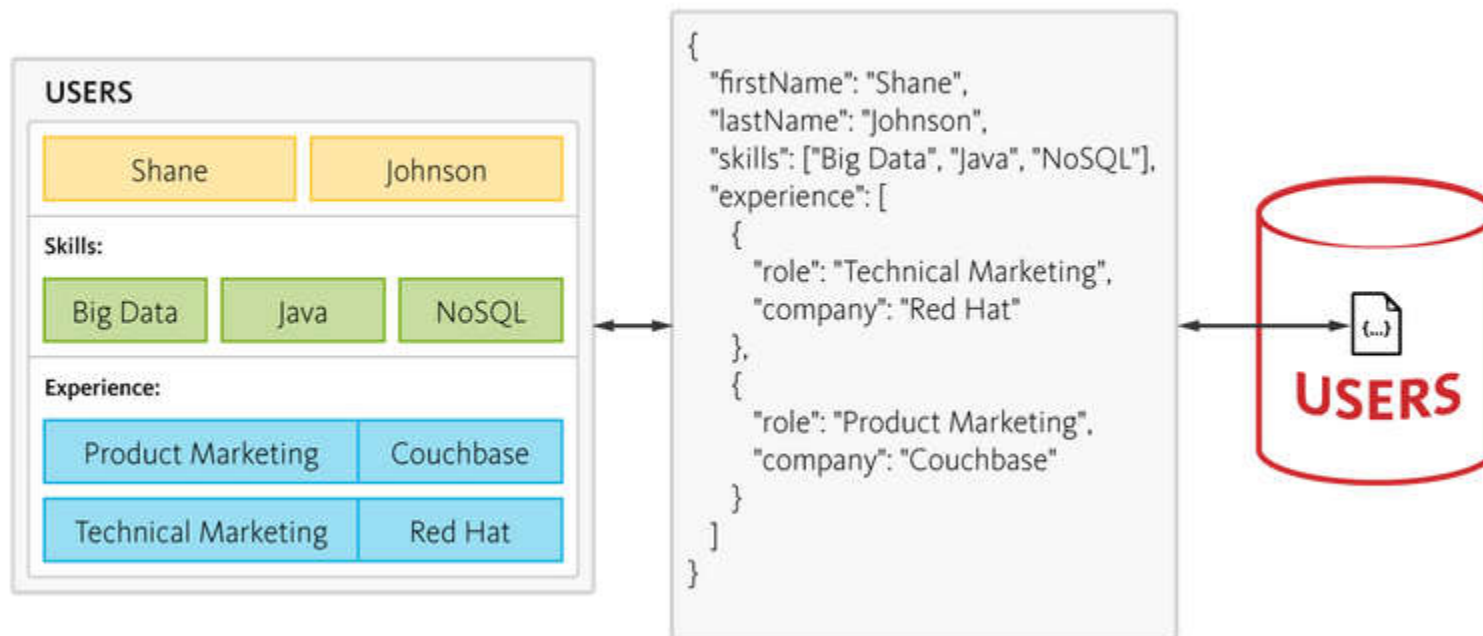
Why nosql? - Use cases where NoSQL will outperform SQL

- **Agile - Simplicity for Easier Development**
- Reading this profile would require the application to read six rows from three table

Shane	Johnson	Big Data	Product Marketing	Couchbase
Shane	Johnson	Big Data	Technical Marketing	Red Hat
Shane	Johnson	Java	Product Marketing	Couchbase
Shane	Johnson	Java	Technical Marketing	Red Hat
Shane	Johnson	NoSQL	Product Marketing	Couchbase
Shane	Johnson	NoSQL	Technical Marketing	Red Hat

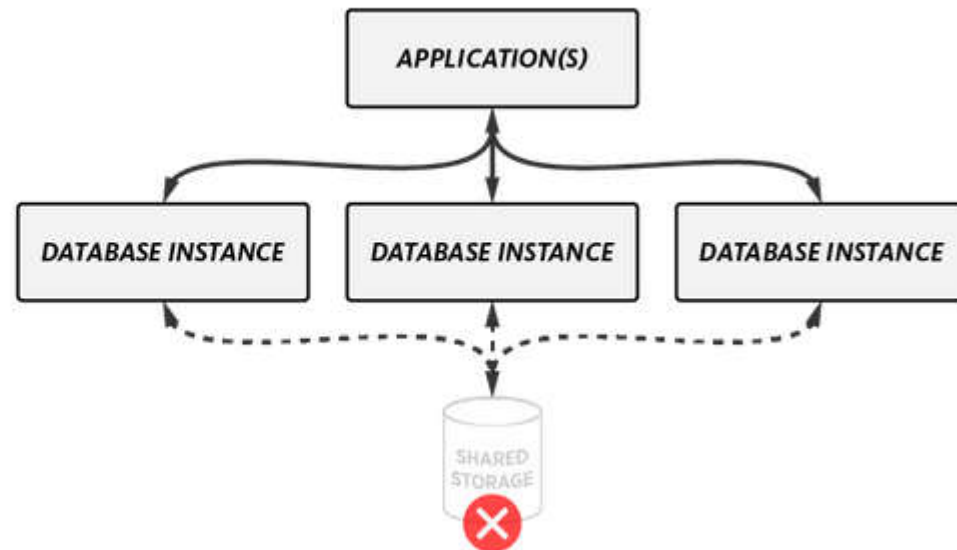
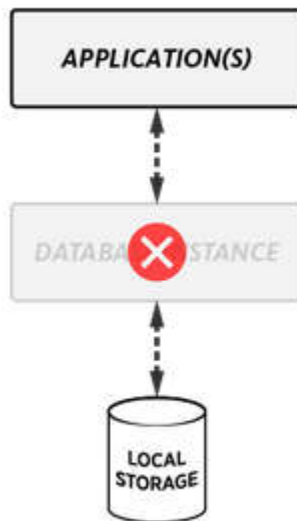
Why nosql? - Use cases where NoSQL will outperform SQL

- **Agile - Simplicity for Easier Development**



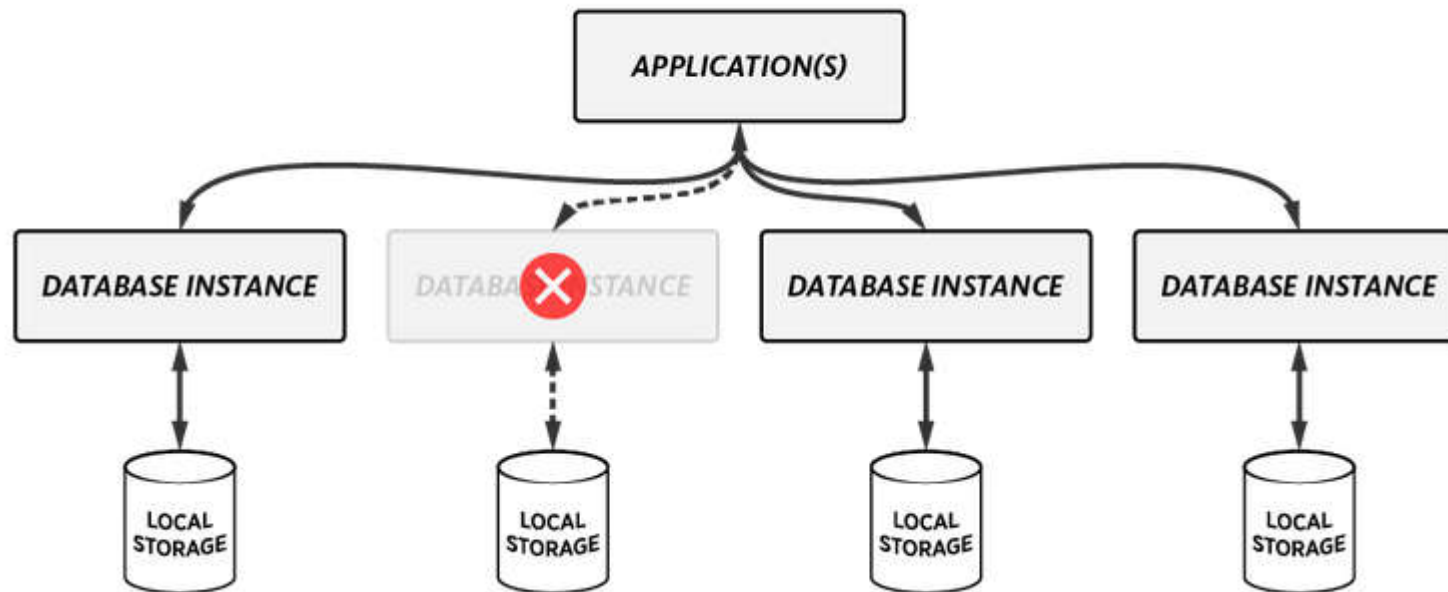
Why nosql? - Use cases where NoSQL will outperform SQL

- Availability for Always-on



Why nosql? - Use cases where NoSQL will outperform SQL

- Availability for Always-on



NOSQL Categories examples - Document-based NOSQL systems

XML is stored into a native XML Type

```
<contact>
  <firstname>Bob</firstname>
  <lastname>Smith</lastname>
  <phone type="Cell">(123) 555-0178</phone>
  <phone type="Work">(890) 555-0133</phone>
  <address>
    <type>Home</type>
    <street1>123 Back St.</street1>
    <city>Boys</city>
    <state>AR</state>
    <zip>32225</zip>
    <country>US</country>
  </address>
</contact>
```

NOSQL Categories examples - Document-based NOSQL systems

- The query retrieves the <Features> child element of the <ProductDescription> element

```
declare @myDoc xml
set @myDoc = '<Root>
<ProductDescription ProductID="1" ProductName="Road Bike">
<Features>
  <Warranty>1 year parts and labor</Warranty>
  <Maintenance>3 year parts and labor extended maintenance is available</Maintenance>
</Features>
</ProductDescription>
</Root>'
SELECT @myDoc.query('/Root/ProductDescription/Features')
```

- Result:

```
<Features>
  <Warranty>1 year parts and labor</Warranty>
  <Maintenance>3 year parts and labor extended maintenance is available</Maintenance>
</Features>
```

NOSQL Categories examples - NOSQL key-value stores

- RIAK as example

```
var search = new RiakSearchRequest
{
    Query = new RiakFluentSearch("famous", "name_s")
        .Search("Lion*")
        .Build()
};

var rslt = client.Search(search);
RiakSearchResult searchResult = rslt.Value;
foreach (RiakSearchResultDocument doc in searchResult.Documents)
{
    var args = new[] {
        doc.BucketType,
        doc.Bucket,
        doc.Key,
        string.Join(", ", doc.Fields.Select(f => f.Value).ToArray())
    };
    Debug.WriteLine(
        format: "BucketType: {0} Bucket: {1} Key: {2} Values: {3}",
        args: args);
}
```

NOSQL Categories examples - NOSQL key-value stores

- The response to a query will be an object contains a list of documents which match the given query.
- The documents returned are Search documents (a set of Solr field/values)

NOSQL Categories examples - column NOSQL systems

- Cassandra as an example
- returns a result-set of rows, where each row consists of a key and a collection of columns corresponding to the query

```
1 cqlsh:demodb> SELECT * FROM emp WHERE empID IN (130,104) ORDER BY deptID DESC;
2 empid | deptid | first_name | last_name
3 -----+-----+-----+-----
4 104 | 15 | jane | smith
5 130 | 5 | sughit | singh
6 (2 rows)
```

NOSQL Categories examples - column NOSQL systems

```
INSERT INTO NerdMovies (KEY, 11924)
              VALUES ('Serenity', 'Nathan Fillion')
              USING CONSISTENCY LOCAL_QUORUM AND TTL 86400;
```

- LOCAL_QUORUM: it's a consistency level type
 - Used in multiple data center clusters.
 - Use to maintain consistency locally (within the single data center).

NOSQL Categories examples - Graph-based NOSQL systems

- Neo4j as an example

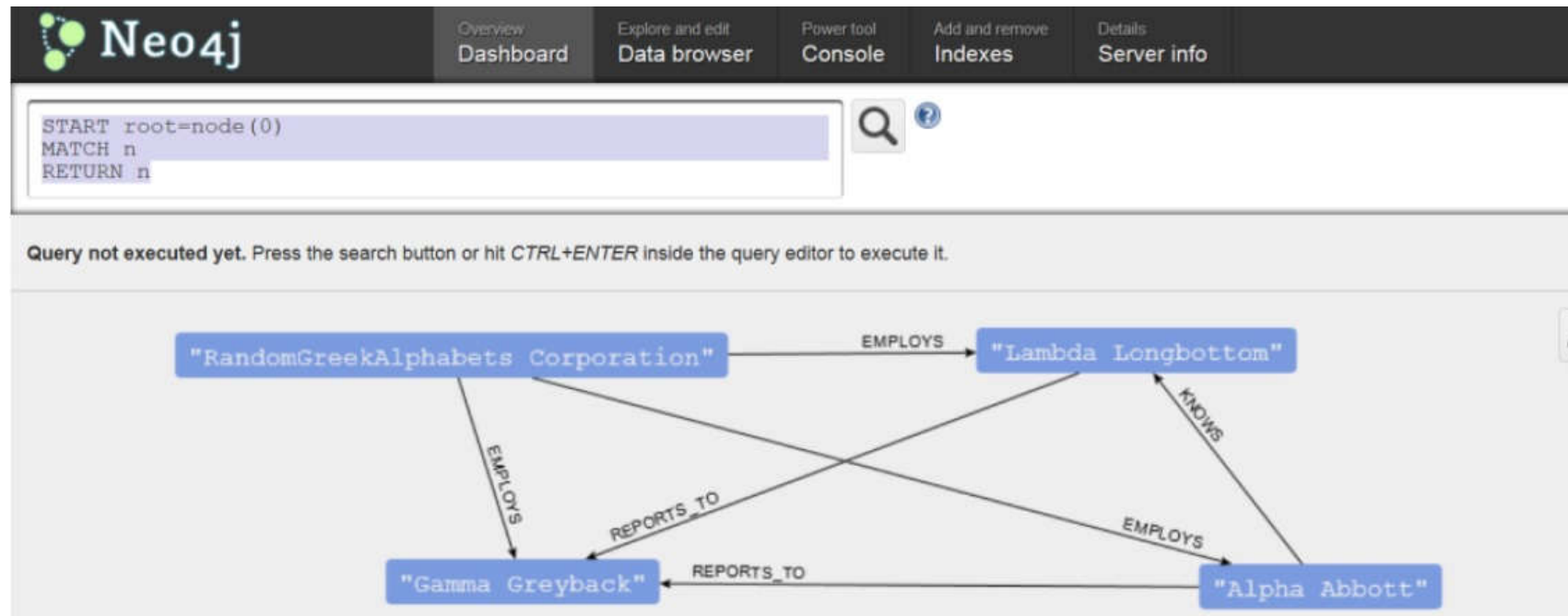
SQL Statement

```
SELECT name FROM Person
LEFT JOIN Person_Department
    ON Person.Id = Person_Department.PersonId
LEFT JOIN Department
    ON Department.Id = Person_Department.DepartmentId
WHERE Department.name = "IT Department"
```

Cypher Statement

```
MATCH (p:Person)<-[:EMPLOYEE]-(d:Department)
WHERE d.name = "IT Department"
RETURN p.name
```

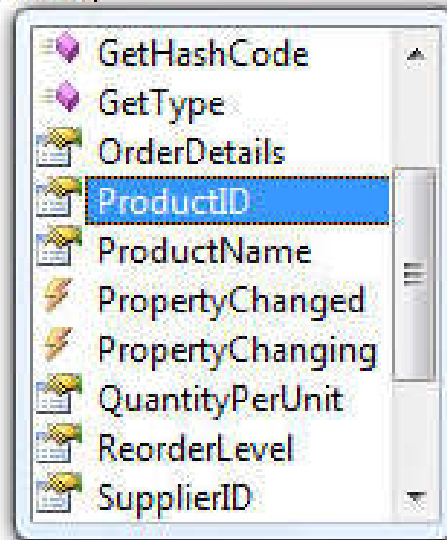
NOSQL Categories examples - Graph-based NOSQL systems



NOSQL Categories examples - Object databases

- LINQ as an example

```
NorthwindDataContext db = new NorthwindDataContext();  
  
var products = from p in db.Products  
                where p.CategoryID == 2  
                select p;  
  
foreach (Product product in products)  
{  
    product.|  
}
```



int Product.ProductID

NOSQL Key-Value Stores

Introduction

DynamoDB Overview

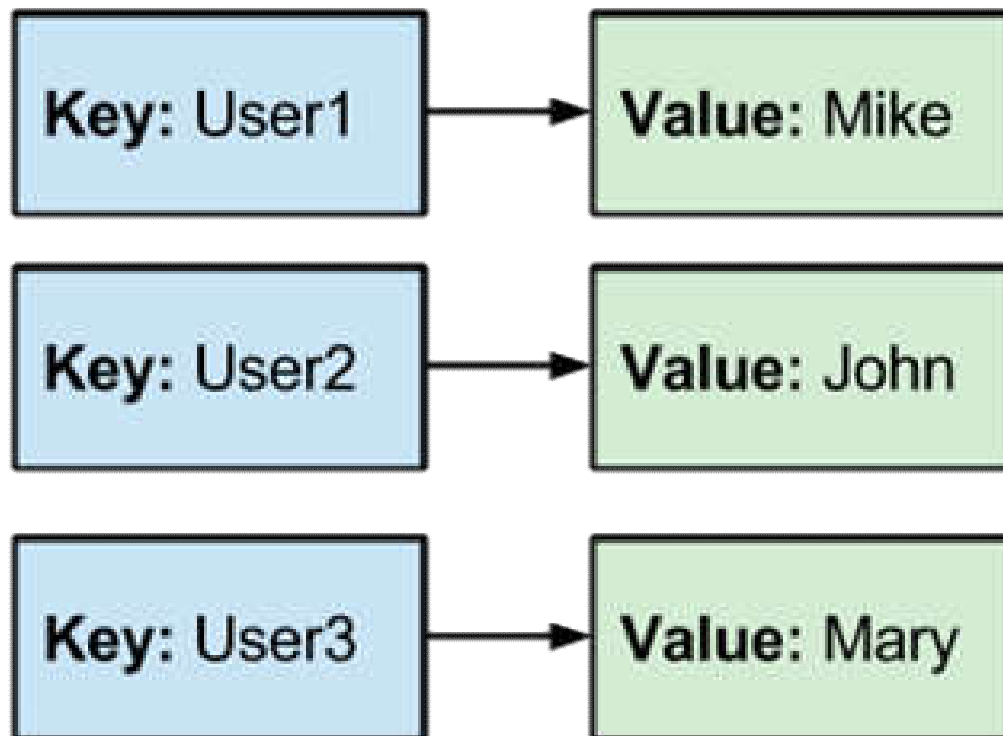
Voldemort Key-Value Distributed Data Store

Examples of Other Key-Value Stores

Introduction

- No query language
- A set of operations that can be used by the application programmers.
- Characteristics:
 - Every value is associated with a unique key.
 - Retrieving the value by supplying the key is very fast.

Introduction



DynamoDB Overview

- Amazon product – part AWS
- Data model is using the concepts of tables, items, and attributes.
 - The table does not have a schema.
 - Holds a collection of self-describing items.
 - The item consist of a number of (attribute, value) pairs
 - Attribute values can be single-valued or multivalued.



DynamoDB



DynamoDB Overview

- Uploads an item to the ProductCatalog table

```
Table table = Table.LoadTable(client, "ProductCatalog");

var book = new Document();
book["Id"] = 101;
book["Title"] = "Book 101 Title";
book["ISBN"] = "11-11-11-11";
book["Authors"] = new List<string> { "Author 1", "Author 2" };
book["InStock"] = new DynamoDBBool(true);
book["QuantityOnHand"] = new DynamoDBNull();

table.PutItem(book);
```

Column-Based or Wide Column NOSQL Systems

- Stores data tables as columns rather than as rows.

Table

Country	Product	Sales
US	Alpha	3,000
US	Beta	1,250
JP	Alpha	700
UK	Alpha	450

Row Store

Row 1	US
	Alpha
	3,000
Row 2	US
	Beta
	1,250
Row 3	JP
	Alpha
	700
Row 4	UK
	Alpha
	450

Column Store

Country	US
	US
	JP
Product	UK
	Alpha
	Beta
Sales	Alpha
	3,000
	1,250
	700
	450

Hbase Data Model and Versioning

- Apache HBase is an open-source, distributed, versioned, non-relational database.
- Column is identified by a combination of (column family:column qualifier).
- Stores multiple versions of a data item, with a timestamp associated with each version.

Hbase Data Model and Versioning

Examples in Hbase. (a) Creating a table called EMPLOYEE with three column families: Name, Address, and Details. (b) Inserting some in the EMPLOYEE table; different rows can have different self-describing column qualifiers (Fname, Lname, Nickname, Mname, Minit, Suffix, ... for column family Name; Job, Review, Supervisor, Salary for column family Details). (c) Some CRUD operations of Hbase.

(a) creating a table:

```
create 'EMPLOYEE', 'Name', 'Address', 'Details'
```

(b) inserting some row data in the EMPLOYEE table:

```
put 'EMPLOYEE', 'row1', 'Name:Fname', 'John'
put 'EMPLOYEE', 'row1', 'Name:Lname', 'Smith'
put 'EMPLOYEE', 'row1', 'Name:Nickname', 'Johnny'
put 'EMPLOYEE', 'row1', 'Details:Job', 'Engineer'
put 'EMPLOYEE', 'row1', 'Details:Review', 'Good'
put 'EMPLOYEE', 'row2', 'Name:Fname', 'Alicia'
put 'EMPLOYEE', 'row2', 'Name:Lname', 'Zelaya'
put 'EMPLOYEE', 'row2', 'Name:MName', 'Jennifer'
put 'EMPLOYEE', 'row2', 'Details:Job', 'DBA'
put 'EMPLOYEE', 'row2', 'Details:Supervisor', 'James Borg'
put 'EMPLOYEE', 'row3', 'Name:Fname', 'James'
put 'EMPLOYEE', 'row3', 'Name:Minit', 'E'
put 'EMPLOYEE', 'row3', 'Name:Lname', 'Borg'
put 'EMPLOYEE', 'row3', 'Name:Suffix', 'Jr.'
put 'EMPLOYEE', 'row3', 'Details:Job', 'CEO'
put 'EMPLOYEE', 'row3', 'Details:Salary', '1,000,000'
```

(c) Some Hbase basic CRUD operations:

Creating a table: `create <tablename>, <column family>, <column family>, ...`

Inserting Data: `put <tablename>, <rowid>, <column family>:<column qualifier>, <value>`

Reading Data (all data in a table): `scan <tablename>`

Retrieve Data (one item): `get <tablename>,<rowid>`

Hbase Data Model and Versioning

- Table is divided into a number of regions.
- Range partitioning.
- Apache Zookeeper and Apache HDFS (Hadoop Distributed File System) are used for management.

Q & A

